fit@hcmus

VNUHCM-UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

COURSE SYLLABUS

CSC13114 – Advanced Web Application Development

1. GENERAL INFORMATION

Course name: Advanced Web Application Development Course name (in Vietnamese): Phát triển ứng dụng web nâng cao

Course ID: CSC13114

Knowledge block: Software Engineering

Number of credits: 4

Credit hours for theory: 45

Credit hours for practice: 30

Credit hours for self-study: 90

Prerequisite: Web Application Development, Introduction to Databases

Prior-course: Web Application Development

Instructors: MSc. NGUYEN HUY KHANH

2. COURSE DESCRIPTION

This graduation course of the Software Engineering major provides advanced knowledge of **building and operating** a multi-component web application: specifying before implementing, designing and evolving the data schema, API contracts, security and authorization, automated quality gates, packaging and releasing with the ability to roll back, and observability and incident handling.

The course **starts where Web Application Development ends**: students can already build a single-page application with authentication, so React basics, REST basics and JWT basics are not taught again here.

In parallel, the course trains the way software engineers work today: **specify first, delegate to an agent, verify with automated guardrails, and keep the decision at the points that cannot be undone**. Students also learn to **put an LLM inside the product itself** (streaming,

tool calling, RAG) and to handle the hardest part of such features: non-determinism, cost, latency, and the consequences of a wrong answer.

3. COURSE GOALS

At the end of the course, students are able to

G1	Work independently or in teams to research and present new software technologies 2.1.1, 2.3.2, 2.4.3
G2	Know and explain English terminology in Software Engineering 1.2.2, 2.4.3, 2.4.4
G3	Present the trends of modern web development 1.5.4
G4	Build, release and operate a real, running multi-component web application 1.5.4
G5	Understand web scalability: cloud computing, NoSQL 1.5.4
G6	Propose and apply appropriate software technologies for a specific project 5.2.1, 5.2.2
G7	Build software projects based on new technologies 2.2, 5.3
G8	Run an agent-assisted development workflow: specify, delegate, build guardrails, verify 5.2.1, 5.3

G9	Integrate LLM components into a web application and control their non-determinism 1.5.4, 5.2.2

4. COURSE OUTCOMES

G1.1	Organize and manage an effective team U
G1.2	Discuss with other team members about topics of the subject U
G1.3	Analyze, synthesize and write technical documents based on predefined templates T, U

G2.1	Explain specialized English terminology of the subject I
G2.2	Extract information and techniques from provided documents and lectures I, T
G3.1	Explain the trends of applications in the Internet environment I, T
G4.1	Design an API contract and evolve it without breaking running clients T, U
G4.2	Design a data schema and run zero-downtime migrations with a rollback path T, U
G4.3	Release and operate: packaging, per-environment configuration, canary, rollback, observability T, U
G4.4 G4.5	Apply coding style and coding conventions U Write unit tests, contract tests and automation tests as quality gates U

G5.1 G5.2	Explain current problems in software development, particularly in web development Apply the latest technologies to solve the problems of modern applications	I, T I, T, U
G6.1 G6.2 G7.1	Self-study and self-develop to acquire new knowledge Analyze, evaluate and select appropriate technologies for a specific application Apply the latest technologies to build highly scalable, multi-platform applications	I, T, U I, T, U I, T, U
G8.1	Write a feature specification tight enough for an agent to implement it correctly without asking back	T, U
G8.2	Build the project harness: convention files, lint, type checking, tests and CI as quality gates	T, U
G8.3	Review agent-generated code for correctness, security, performance and licensing	T, U
G8.4	Place human gates in the right places: only at irreversible actions	I, T, U
G8.5	Configure least privilege for the tools an agent is allowed to call	I, T
G8.6	Record and honestly disclose the work done by AI	U
G9.1	Call LLMs from the backend: streaming, tool calling, context management	T, U

G9.2	Build knowledge retrieval (RAG) over the application's own

	data	T, U
G9.3	Write an eval suite for a non-deterministic feature and read its results statistically	T, U
G9.4	Design guardrails: hard limits, human confirmation, behaviour when the model is wrong	I, T, U
G9.5	Estimate and control the cost, latency and quotas of an LLM feature	I, T

G8.4 and G9.4 are the focus of grading. Judgement about boundaries matters more than technical polish: a student who knows where **not** to let the machine decide is an engineer.

5. TEACHING PLAN THEORY

1	Introduction. Fromvibe coding to vibe engineering. The course's AI-use policy	G3.1, G8.6	Lecturing Team formation
2	Specifications for agents: from user story to an executable spec; where specs are always incomplete	G8.1	Individual spec writing Specs swapped: another student implements each one IA#1, PA#1
3	Harness and working environment: convention files, tools, MCP, least privilege, per-environment configuration	G8.2, G8.5	Demonstration: building a harness on a real repository Submit on Google Classroom
4	Data: modelling, indexes, transactions, zero-downtime migration, backup and restore	G4.2	Changing the schema of a running system, then rolling back IA#2

5	API contracts: unified error model, pagination,	G4.1	Contract agreed first; an agent generates the implementation Submit on Google Classroom

	idempotency, versioning		
6	Security: owner-based authorization and RBAC, secrets management, supply chain, prompt injection	G4.1, G8.3	Red-teaming another team's code Submit on Google Classroom
7	Quality gates: the testing pyramid, contract tests, Playwright, CI. AI reviewing AI	G4.5, G8.2, G8.3	Building a CI pipeline that blocks merges when a gate is red PA#2
8 9 10	Packaging and release: Docker, per-environment configuration, migration vs. deploy ordering, canary, rollback, feature flags Putting an LLM into the product: calling models, streaming, tool calling, RAG Non-determinism: evals, guardrails, human gates, cost and latency	G4.3, G5.1, G5.2 G9.1, G9.2 G9.3, G9.4, G9.5, G8.4	Releasing a change, then rolling it back within 5 minutes IA#3 Demonstration Hands-on practice PA#3 Writing evals Red-teaming guardrails IA#4, PA#4
11	Operations: logs, metrics, traces, alerting, SLOs, incidents and post-mortems, queues, retries, rate limiting, cost	G4.3, G5.1, G6.1	Reading the trace of a real incident PA#5 Writing a post-mortem

The eleven topics read as one line: **specification → guardrails → data → contracts → security → quality gates → release → LLM → non-determinism → operations.**

Optimisation is covered in the seminar, not in the lectures. Optimisation only makes sense once a system runs and can be measured, so it does not take a lecture slot. Each team takes one topic, presents it, and another team red-teams it; this is the SEMINAR component (7%) in the Assessments section. Topic list:

- Rendering and performance: CSR/SSR/SSG/streaming, RSC, the cost of hydration, Core Web Vitals
- Caching and CDN: cache layers, invalidation, headers
- Realtime at scale: scaling WebSocket, backpressure, offline-first
- Monorepos and design systems
- Vector databases for RAG: chunking, hybrid search, measuring retrieval quality ● Optimising model cost: model selection, prompt caching, batching, quotas **Note on topic 10.** This is the hardest and newest topic, and the one that separates students most clearly. For their own team's LLM feature, students must be able to answer three questions:

- **Who pays when it is wrong?** Whom a wrong output harms, how badly, and whether it can be undone.
- **How do we know it is wrong?** How many cases the eval suite has, what the pass threshold is, who runs it.
- **Where do we stop it?** Which action the human gate sits on, and why it does **not** sit on the others.

6. LABORATORY

1	Setting up the repository and harness: conventions, lint, format, tests, CI	G8.2	PA#2 Guided hands-on setup Q&A

2 3	Writing a feature spec; another student implements it from that spec Data schema and expand/contract migration on a running system	G8.1 G4.2	Spec exchange between students Discussion IA#1 Hands-on practice IA#2 Q&A

4	API contract: error model, pagination, idempotency	G4.1	Hands-on practice Submit on Google Classroom Discussion
5	Cross-team security review: authorization, secrets, dependencies, prompt injection	G4.1, G8.3	Cross-team review Submit on Google Classroom Discussion
6	Quality gates: contract tests, Playwright, CI pipeline	G4.5, G8.2	PA#2 Hands-on practice Q&A
7 8 9 10	Dockerizing, per-environment configuration, release and rollback Calling an LLM fromthe backend: streaming and tool calling RAG over the application's own data Seminar: each teampresents its optimisation topic; another team red-teams it	G4.3, G5.1 G9.1 G9.2 G1.1–G1.3, G9.3	Hands-on practice IA#3 Demonstration Demonstration PA#3 Hands-on practice Hands-on practice PA#3 Q&A Team presentations SEMINAR Red-teaming by another team

7. ASSESSMENTS

Overall structure set by the lecturer: **theory exam 40% · course project 40% · assignments 20%.**

	I Theory exam: final, no AI	**Closed-book written exam taken in the exam room. Three parts:** **A. Foundations of a running system (30%): API contracts, data schema and migration, security and authorization,**	**40%**

PA	**Course project: teams of atmost 3 students**	**packaging and release, observability** **B. Reviewing a specification and designing the harness (35%): what the spec is missing, where the gates go** **C. Non-determinism (35%): for an LLM feature, who is at risk, how to evaluate it, where to stop it. An answer of "a human gate at every step" <u>does not get full marks.</u>**	**40%**
P A # 1	Proposal and plan	Users and problem, the LLM feature and the cost of it being wrong, scope, a six-milestone plan, risks, technology choices. No spec attached. G1.1, G6.2	4%

PA#2	Harness and quality gates	Convention files, lint, tests, and a CI that actually blocks merges	G4.5, G8.2, G8.5	4%
PA#3	Prototype with LLM integration	One LLM feature running for real inside the product	G4.3, G7.1, G9.1, G9.2	7%
PA#4	Eval, guardrails and human gate	Eval suite, guardrails, and the red-team report from another team	G9.3, G9.4, G8.4	7%
PA#5	Finalisation and operations	The final version running, with observability and one incident post-mortem	G4.3, G7.1, G8.6	3%

PA#6	Individual oral examination	Answering questions on the team's own code and the team's own decisions	G3.1, G4.1, G8.3, G8.4	15%
BT	**Assignments**			**20%**
IA#1	Specifying a feature	Another student implements the feature from this spec; the number of questions they have to ask back is also graded	G8.1	3%

IA# 2	Zero-downtime migration	Change the schema of a running system using expand/contract and demonstrate the rollback path	G4.2	3%
IA# 3	Release and rollback	Docker, per-environment configuration, canary then rollback; logs submitted with the work	G4.3, G5.1	3%
IA# 4	Eval	An eval suite for an LLM feature, with a commentary on the results	G9.3	4%
SE MI NA R	Seminar	One optimisation topic from the list in the Teaching Plan; another team red-teams it	G1.2, G1.3, G2.2	7%

Total: 40 + 40 + 20 = **100%**. Within the assignments: **IA 13%** (done at home, AI allowed, an AI log is required) and **SEMINAR 7%**.

Assessment codes. IA#n: *individual assignment*, done alone; GA#n: *group assignment*; PA#n: *project assignment*, the project milestones; SEMINAR; THI (theory exam). This coding continues that of Web Application Development, which students have just taken (IA01–IA07, GA01–GA04).

Submission file name. One single zip file: <StudentID>_<self-assessed score>.zip for individual work, <StudentID1>-<StudentID2>-<StudentID3>_<score>.zip for group work, with student IDs in ascending order and the score on the rubric's 100-point scale. A wrongly named file is not penalised but is graded later, because the grading script matches submissions to the enrolment list by file name.

SELF_ASSESSMENT_REPORT.md is required inside the zip: a self-grading against each rubric criterion, each line with evidence pointing to a file, section, commit or test name, adding up exactly to the score in the file name, plus a section on *what was not achieved*.

Honesty deduction. The self-assessed score is compared with the actual score, the difference measured on the 100-point scale: within ±10, no deduction; self-assessed higher than actual by 11–20, minus 3 points; by 21–30, minus 6; by more than 30, minus 10; self-assessed lower than actual by 21 or more, minus 3. A missing SELF_ASSESSMENT_REPORT.md costs 10 points and the score in the file name is ignored. A self-graded criterion without an evidence line counts as claimed but not done. The score never goes below 0.

Every assessment has a rubric, published at the same time as the assignment and attached directly to the assignment on Google Classroom. Rubrics are scored out of 100 and then converted to the weights in the table above; every criterion must be observable in the submission.

8. RESOURCES

Textbooks

- Alex Banks, Eve Porcello. *Learning React: Functional Web Development with React and Redux*. O'Reilly Media, 2017.
- Greg Lim. *Beginning Node.js, Express & MongoDB Development*. 2019. ● Roberto Infante. *AI Agents and Applications With LangChain, LangGraph, and MCP*. Manning Publications, 2026. A copy is available in the Faculty's document collection.

Others

- Boris Cherny. *Programming TypeScript*. O'Reilly Media, 2019.
- Sam Newman. *Building Microservices*. O'Reilly Media, 2015.
- Faculty seminar materials: *From Vibe Coding to Vibe Engineering*; *Agentic Engineering Workflow*.
- The official documentation of the model provider chosen by the team, latest version.

- React Documentation (https://react.dev); Model Context Protocol (https://modelcontextprotocol.io).

9. GENERAL REGULATIONS & POLICIES

- All students are responsible for reading and following strictly the regulations and policies of the school and university.
- Students who are absent for more than 3 theory sessions are not allowed to take the exams.
- For any kind of cheating and plagiarism, students will be graded 0 for the course. The incident is then submitted to the school and university for further review. ● Students are encouraged to form study groups to discuss on the topics. However, individual work must be done and submitted on your own.

AI use policy

The same as in CSC13008 Web Application Development, and stricter on two points because this is a graduation course.

- **Disclosure.** Every project milestone includes an AI-LOG.md: the tools, the scope, and the places that were changed from the machine-generated version. ● **Accountability.** "The AI wrote it" is not an excuse for any defect.
- **The theory exam (40%) and the oral examination (15%) are both supervised.** Any part of the project that a student cannot explain earns no marks, even if it works correctly.
- **No AI in the theory exam:** closed-book, taken in the exam room.
- **No secrets or personal data in prompts.** A project that integrates an LLM must state which data is sent to external services and why that is acceptable. ● **LLM components in the product must have guardrails.** A feature that calls a model and displays the result directly, with no eval, no limits and no human confirmation, does **not** count as completing PA#3, however well the demo runs.